Event-Driven & Reactive Systems

Mastering Software Architecture Patterns and System Design

Rahul Rajat Singh

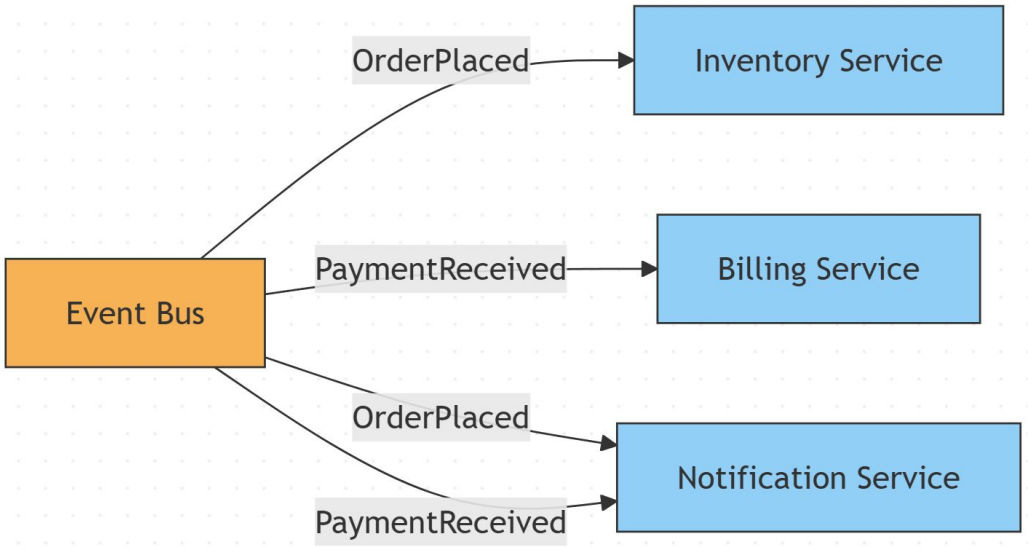
Event-Driven Architecture Basics

Event-Driven & Reactive Systems

Mastering Software Architecture Patterns and System Design

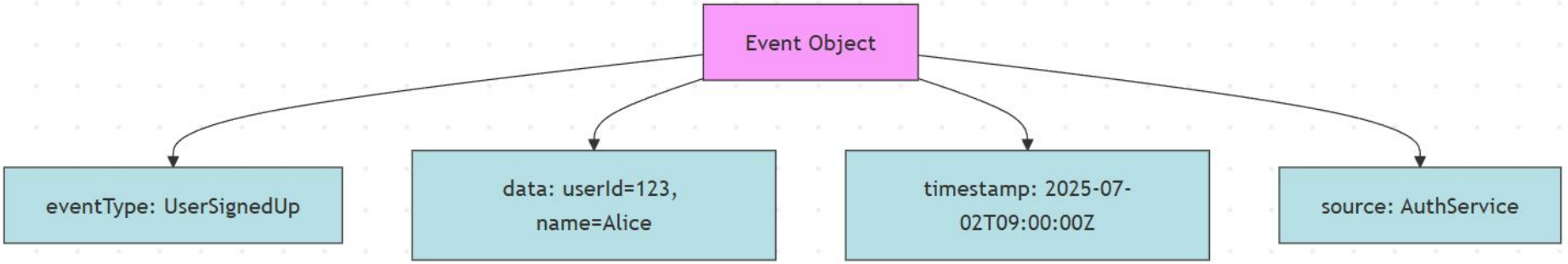
Introduction to Event-Driven Architecture

- Software design paradigm centered around events
- Promotes loose coupling and h scalability
- Enables asynchronous communication between components



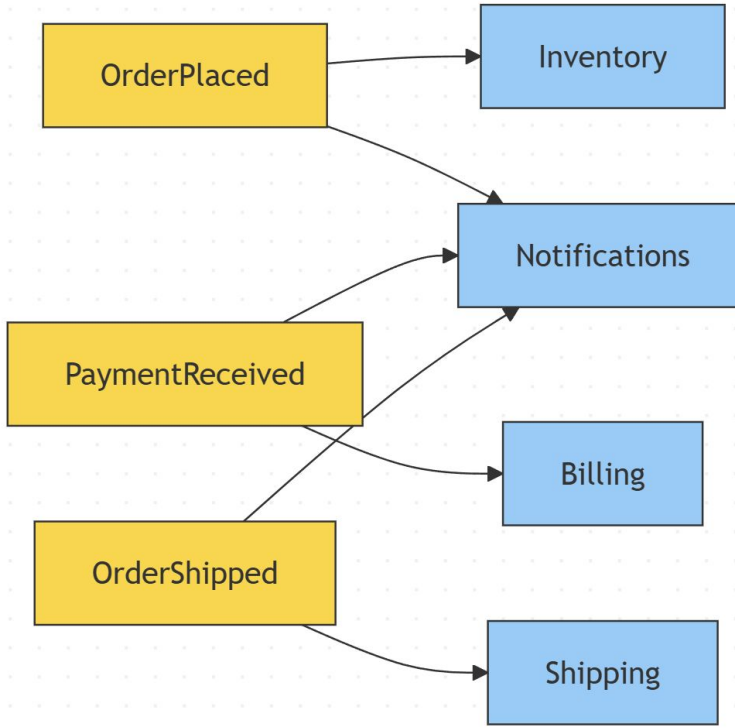
What is an Event?

- Represents a significant state change (e.g., "UserSignedUp")
- Immutable and descriptive
- Contains metadata (timestamp, ID, source)



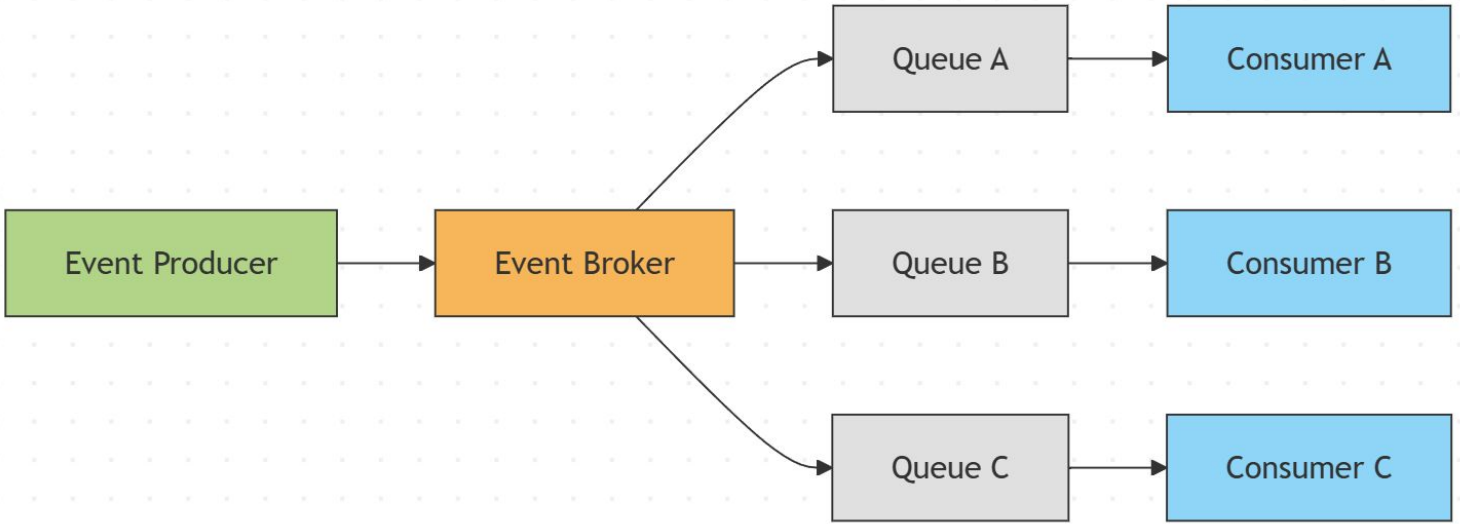
Events as First-Class Citizens

- Events drive logic, workflows, and data flows
- Stored, versioned, and replayable
- Central to system design, not just triggers



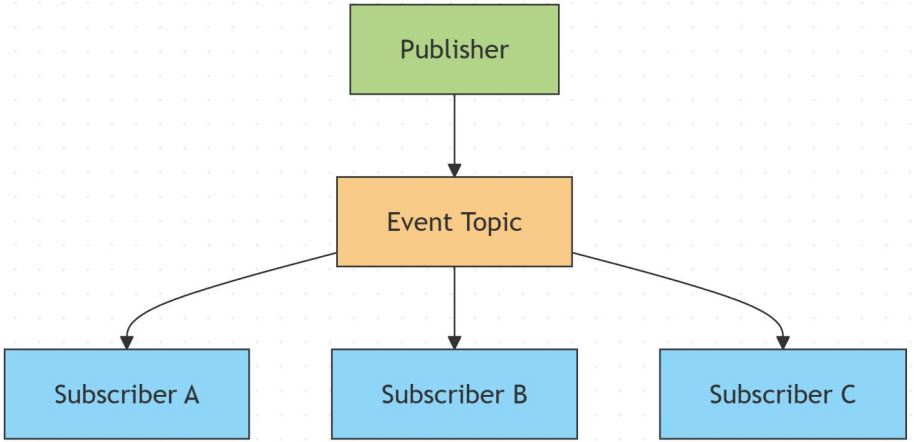
Asynchronous Flow in Action

- Producers emit events without waiting
- Consumers handle events at their own pace
- Improves scalability and resilience



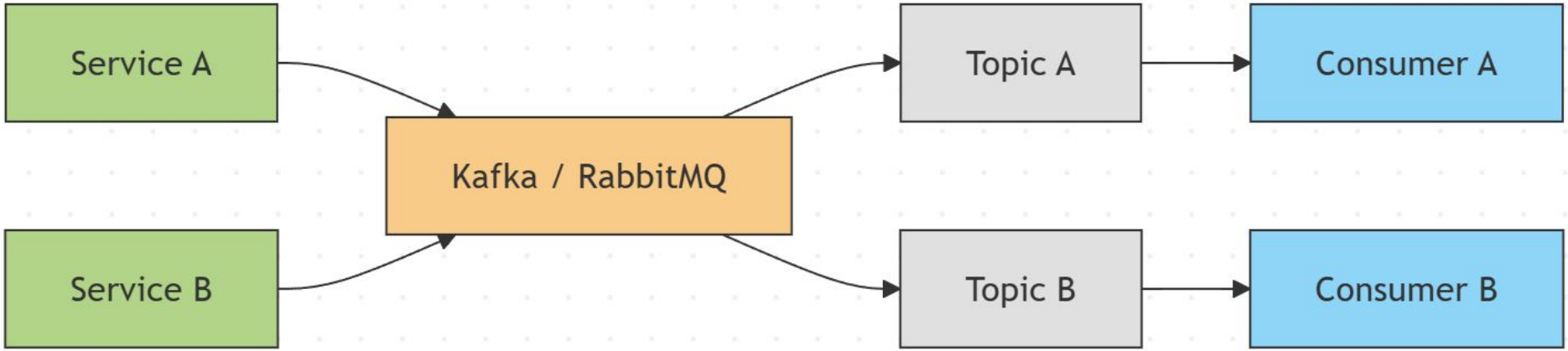
Publish-Subscribe (Pub-Sub) Pattern

- Events published to a topic/channel
- Multiple subscribers receive relevant events
- Promotes decoupling between senders and receivers



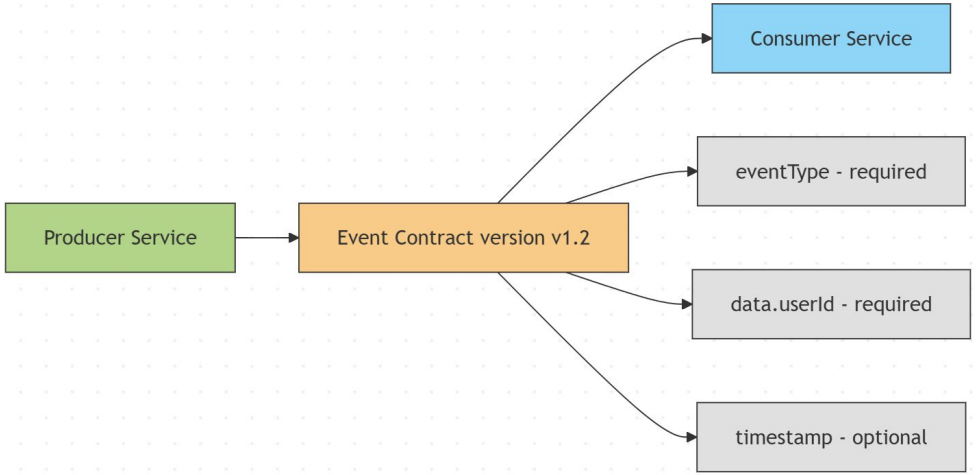
Event Brokers and Infrastructure

- Middleware that routes events (e.g., Kafka, RabbitMQ)
- Handles delivery, retries, and message durability
- Often decouples services via topics or queues



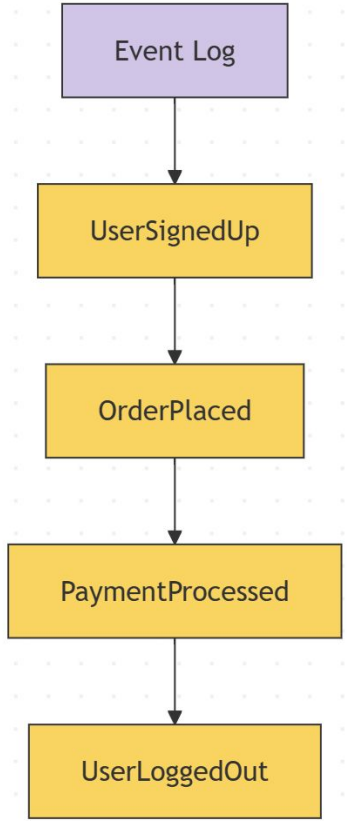
Event Contracts

- Schema defining the structure of an event
- Enables versioning and validation
- Critical for compatibility between services



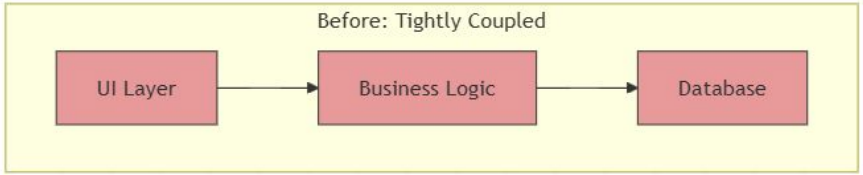
Auditability & Traceability

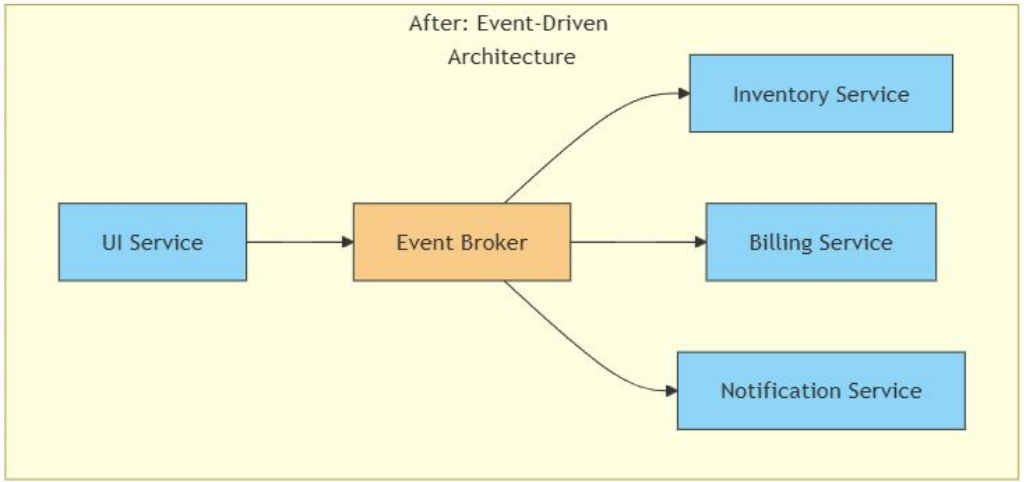
- Events create a historical log of system behavior
- Useful for debugging, compliance, and replay
- Events can be persisted in log stores or event stores



Benefits of Event-Driven Architecture

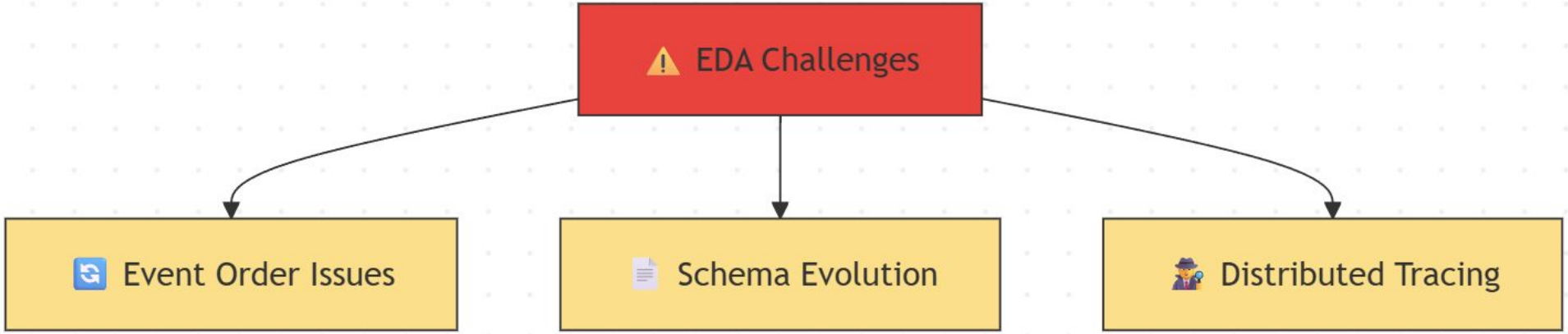
- Loose coupling between components
- High scalability and fault tolerance
- Enables real-time responsiveness





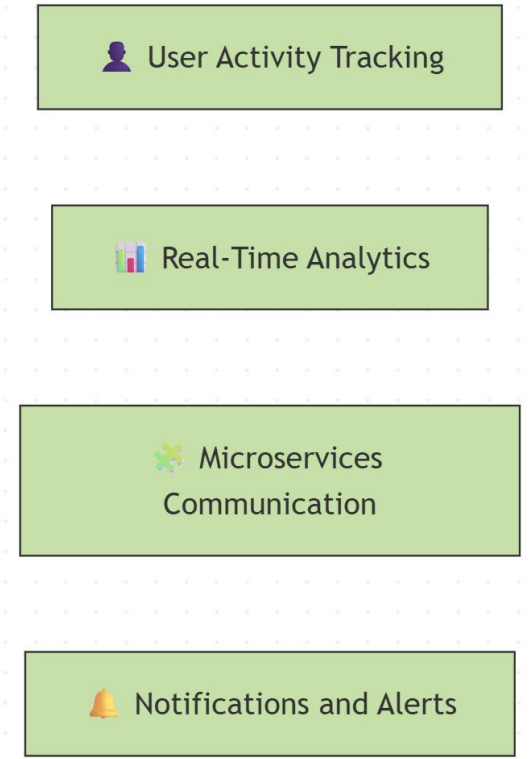
Challenges and Trade-Offs

- Complexity in event flow tracking
- Handling event ordering and duplicates
- Designing consistent event schemas



Common Use Cases

- User activity tracking
- Real-time analytics and dashboards
- Microservices communication
- Notification and alert systems



Summary - Event-Driven Architecture Basics

- Events represent facts — immutable, traceable, and central to system design
- Event-Driven Architecture enables loose coupling, asynchronous flows, and scalable systems
- Key components: Events, Publishers, Subscribers, Brokers, and Contracts
- Supports auditability, real-time responsiveness, and flexible integration
- Requires careful design: schema evolution, observability, and event ordering
- What's next:
  - Event Sourcing + CQRS

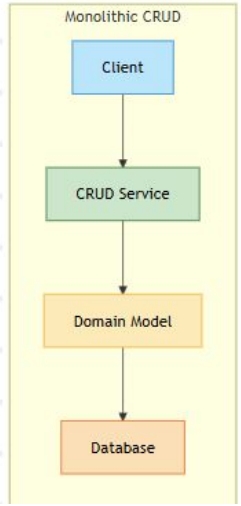
Event Sourcing + CQRS

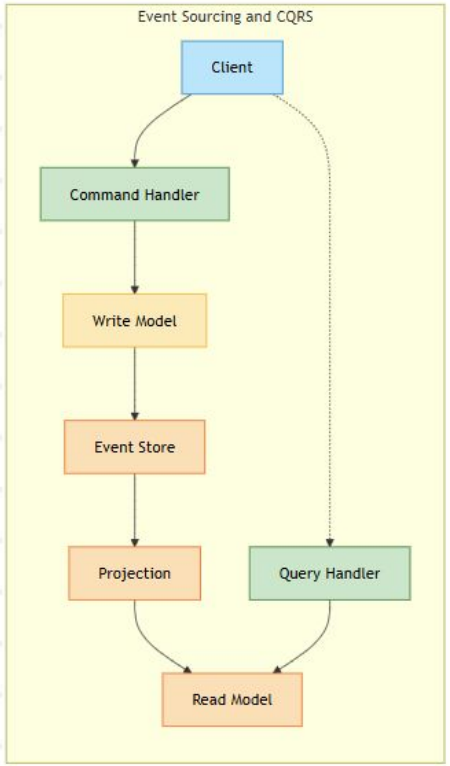
Event-Driven & Reactive Systems

Mastering Software Architecture Patterns and System Design

Why Event Sourcing + CQRS?

- Modern systems demand scalability, auditability, and performance
- Traditional CRUD models can mix responsibilities, reducing flexibility
- Event Sourcing captures every state change as an event
- CQRS splits command (write) and query (read) responsibilities for clarity and optimization





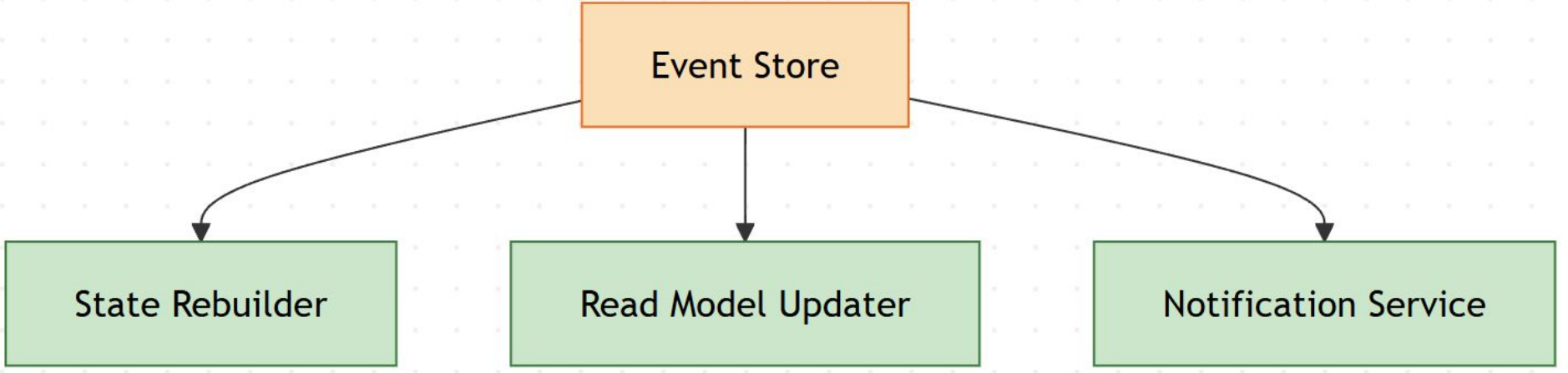
What is Event Sourcing?

- State is rebuilt by replaying a sequence of past events
- Events are immutable and append-only
- Event log is the source of truth, not a database record
- Enables full audit trails and historical debugging



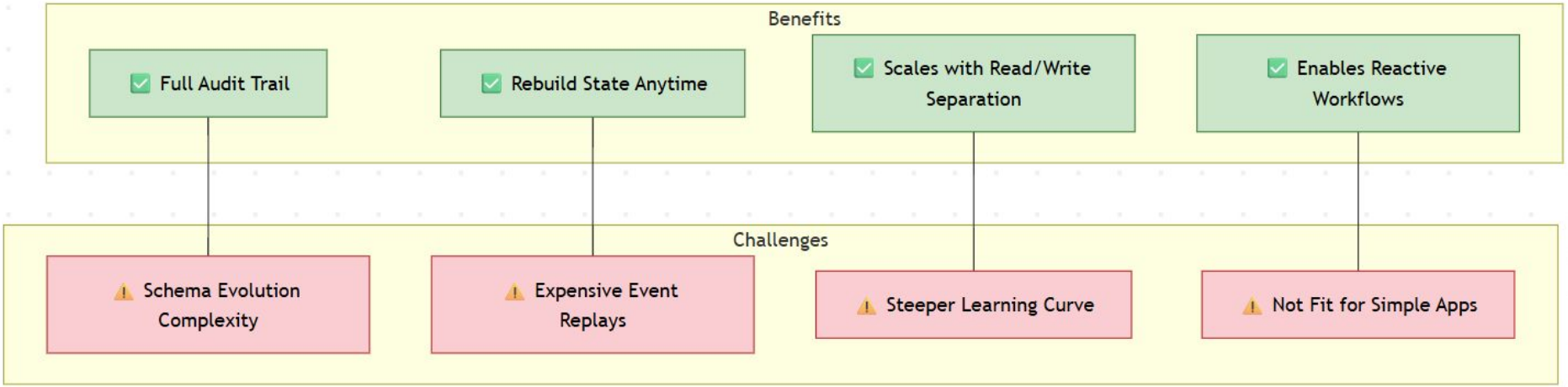
Benefits of Event Sourcing

- Complete audit trail and debugging history
- Easy to rebuild state for new projections or bug fixes
- Natural fit for distributed systems and DDD
- Enables reactive patterns (e.g., publish events to other systems)



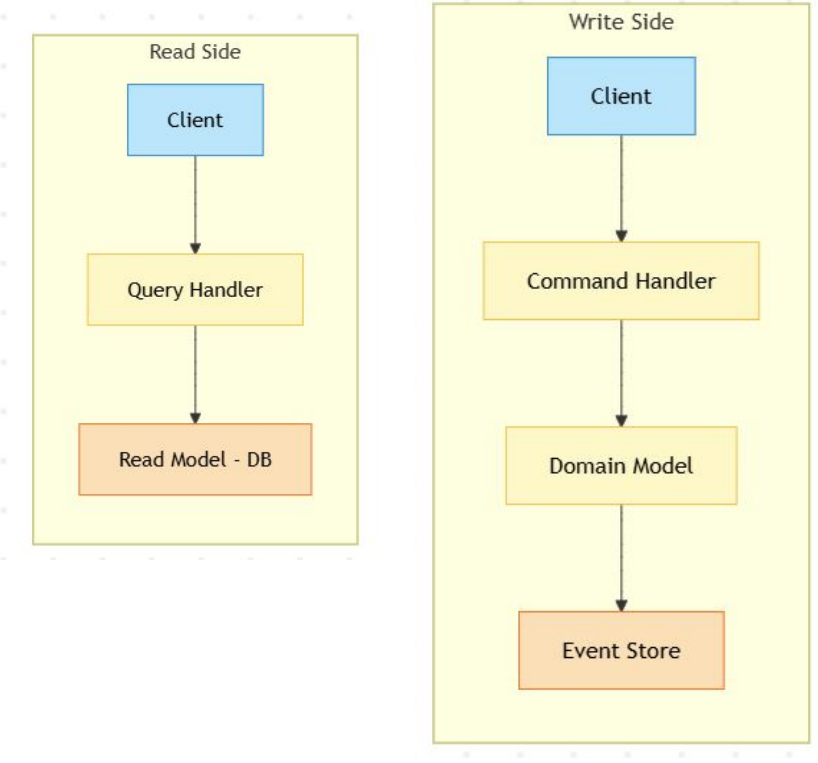
Challenges in Event Sourcing

- Event schema evolution and versioning complexity
- Replaying large event logs can be expensive
- Debugging and mental model complexity
- Not ideal for all use cases (e.g., simple CRUD apps)



What is CQRS (Command Query Responsibility Segregation)?

- Commands = write operations (intent to change state)
- Queries = read operations (no side effects)
- Separation allows independent optimization
- Encourages clear boundaries and scalability



CQRS in Practice – Write Model

- Handles domain logic and business rules
- Emits events upon state changes
- Events are persisted in the event store
- Commands are validated and processed synchronously



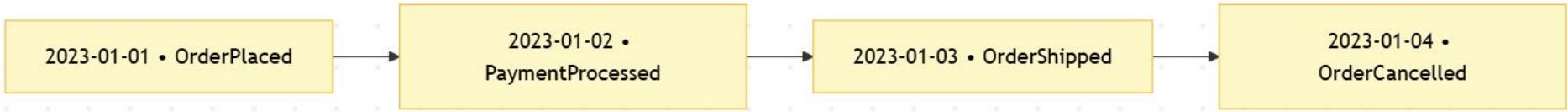
CQRS in Practice – Read Model

- Built from projected events for fast querying
- Can use optimized schemas (denormalized, indexed)
- Can have multiple views for different use cases
- Eventual consistency with write model



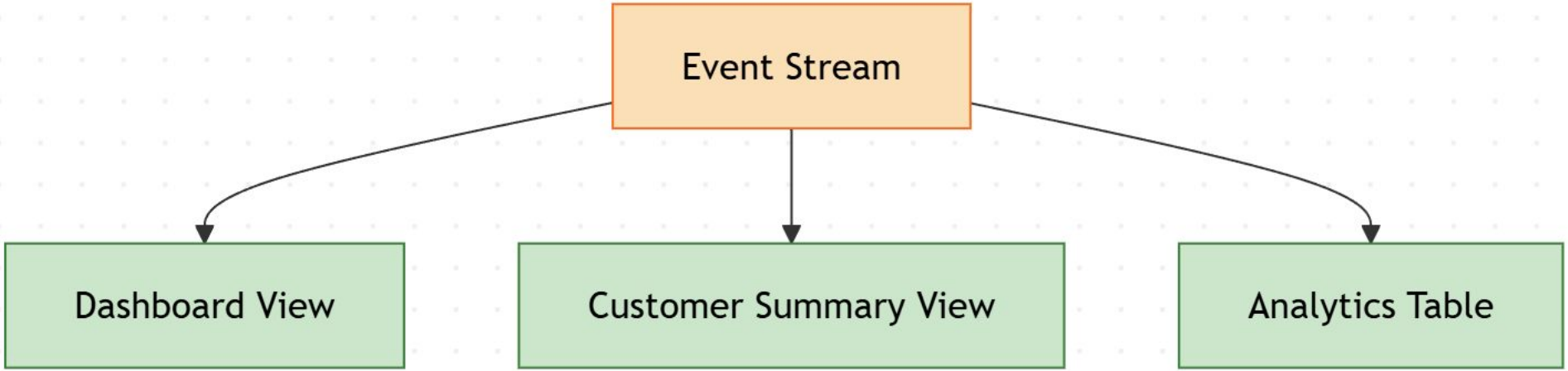
Event Store – The Heart of the System

- Central log of all domain events
- Append-only, versioned, and timestamped
- Enables rebuilding state and projections
- Supports auditing, replay, and debugging



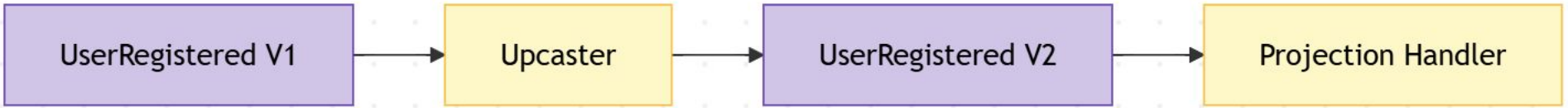
Projections – Building Read Models

- Transform events into queryable views
- Can be rebuilt anytime by replaying events
- Multiple projections can be derived from the same event stream
- Supports polyglot persistence and custom schemas



Event Versioning Strategies

- Use version numbers in event types (e.g., UserRegisteredV1, UserRegisteredV2)
- Evolve schema by adding non-breaking fields
- Write upcasters to transform older events at runtime
- Maintain backward compatibility where possible



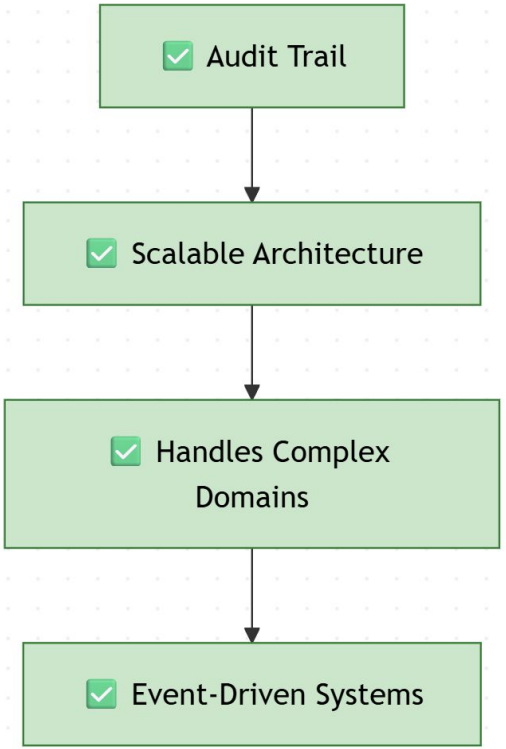
Putting It Together – Full Flow

- Command triggers change in write model
- Event is persisted and published
- Projections update read models
- Queries fetch from optimized stores



When to Use Event Sourcing + CQRS

- Domains with complex logic and audit needs
- Systems with high read/write asymmetry
- Use cases demanding traceability or eventual consistency
- Event-driven microservices architectures



Summary – Event Sourcing + CQRS

- Event Sourcing = storing events, not state
- CQRS = separating read and write paths
- Together: scalable, auditable, flexible systems
- Key challenges: complexity, versioning, eventual consistency
- What's next:
  - Reactive Principles & Reactive Systems

Reactive Principles & Reactive Systems

Event-Driven & Reactive Systems

Mastering Software Architecture Patterns and System Design

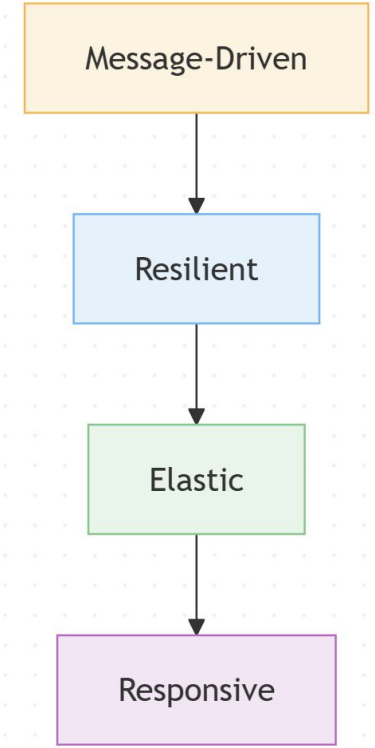
What is a Reactive System?

- A system designed to handle varying loads, failures, and latency
- Prioritizes responsiveness and user experience
- Event-driven and non-blocking by design
- Promotes loose coupling and system isolation



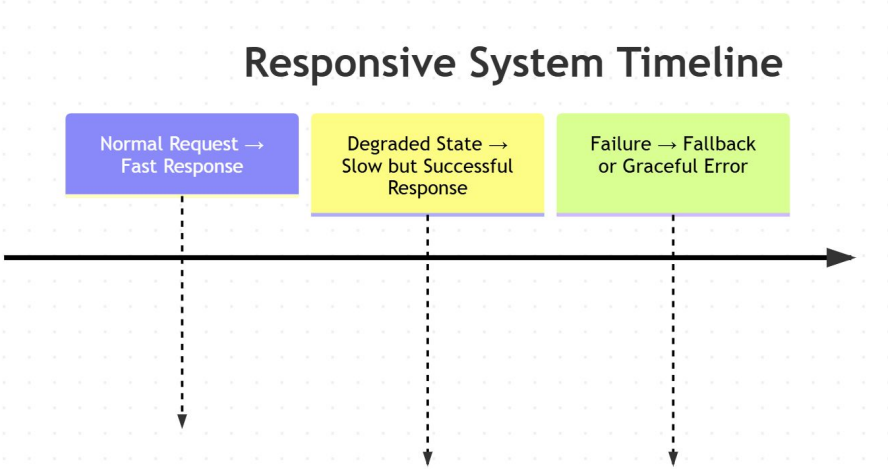
The Reactive Manifesto: An Overview

- Introduced to formalize principles for modern software systems
- Inspired by the needs of distributed, cloud-native environments
- 4 Pillars: Responsive, Resilient, Elastic, Message-Driven
- Chain of dependency: Each principle enables the next



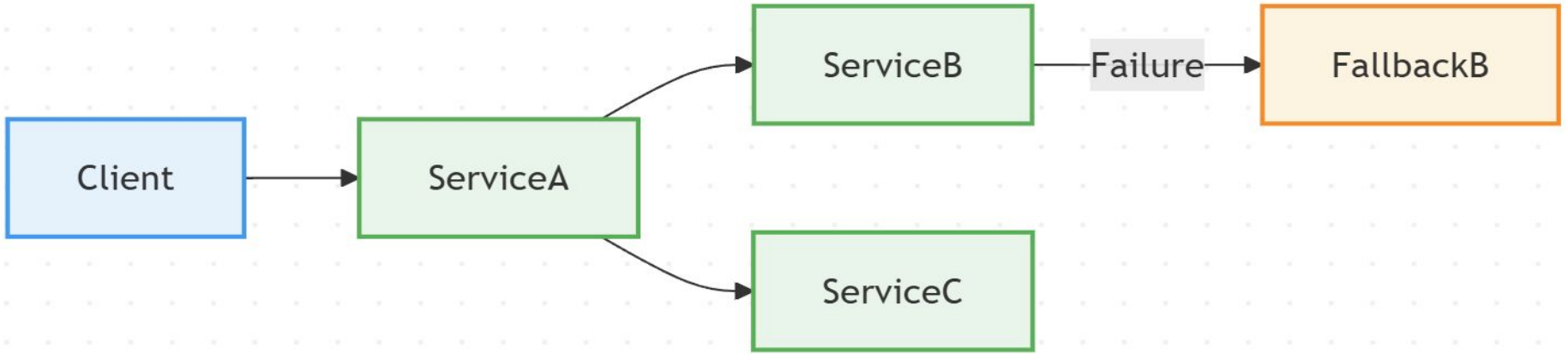
Principle 1: Responsive Systems

- Always respond in a timely, consistent way
- Builds user trust and usability
- Propagates responsiveness throughout the system
- Depends on failure handling and backpressure



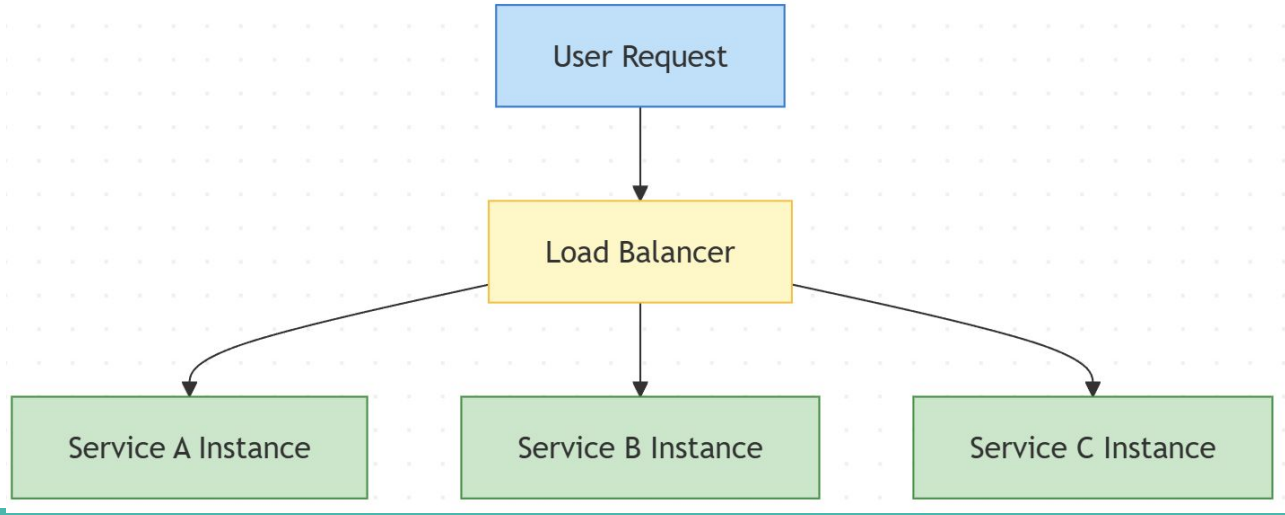
Principle 2: Resilient Systems

- Recover from failure and remain responsive
- Failure is expected and isolated
- Built on supervision, retries, and fallbacks
- Uses circuit breakers and self-healing design



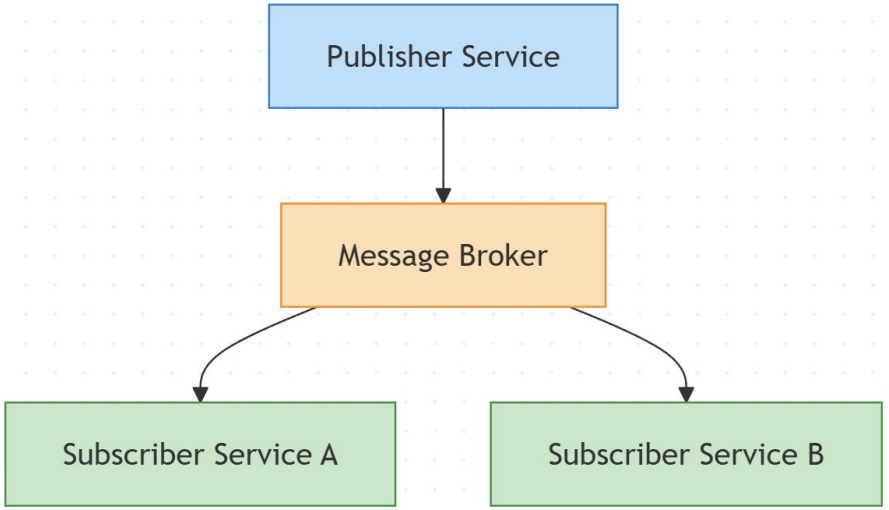
Principle 3: Elastic Systems

- Adapt to changes in workload
- Horizontal scaling, autoscaling
- Use of stateless components
- Resource-aware load balancing



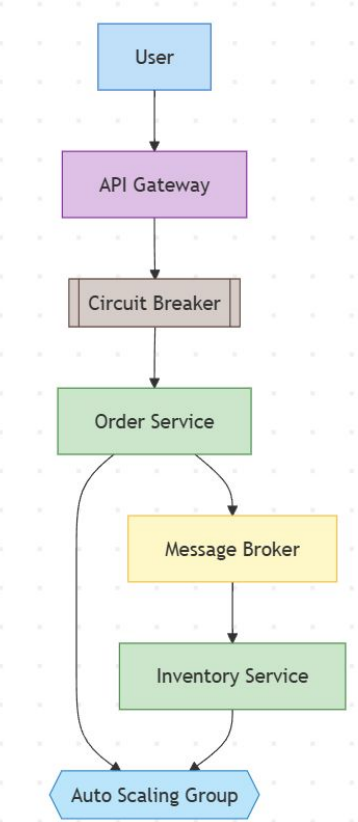
Principle 4: Message-Driven Systems

- Asynchronous message-passing is the backbone
- Enables location transparency & decoupling
- Supports elasticity and resilience
- Supports non-blocking, backpressured communication



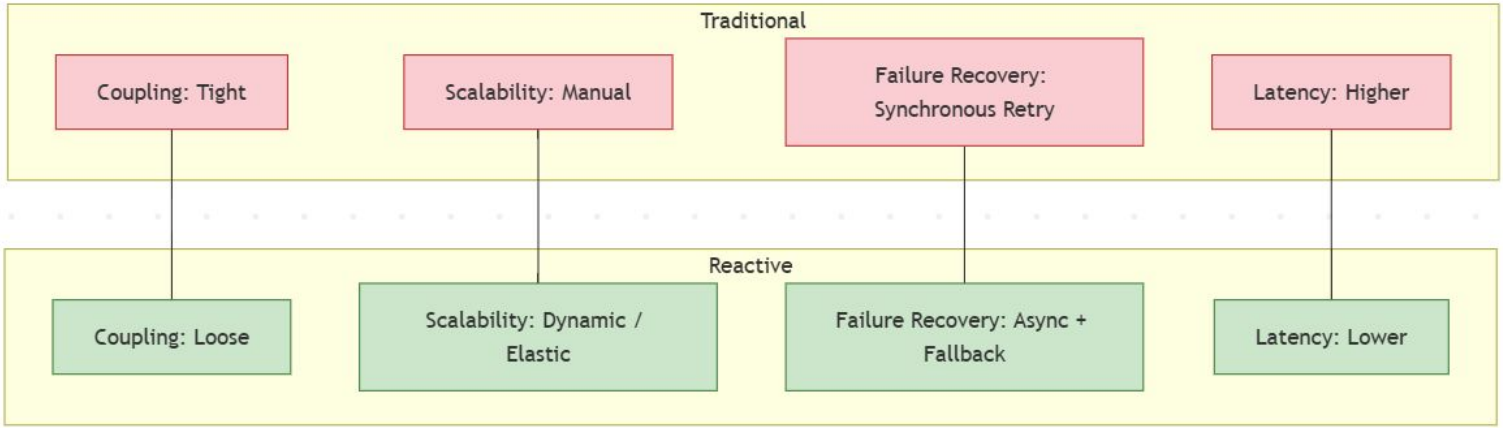
Putting It Together: Reactive System Architecture

- Combine all four principles cohesively
- Services isolate state and fail independently
- Messages ensure async flow and elasticity
- System reacts to load and failures predictably



Reactive vs Traditional Systems

- Traditional: synchronous, blocking, tightly coupled
- Reactive: asynchronous, non-blocking, loosely coupled
- Reactive enables higher resilience and scalability
- Cost: complexity and rethinking architecture



When to Use Reactive Systems

- High-concurrency applications (e.g., messaging, streaming)
- Real-time systems needing low latency
- Microservices and distributed systems
- When downtime or lag is unacceptable



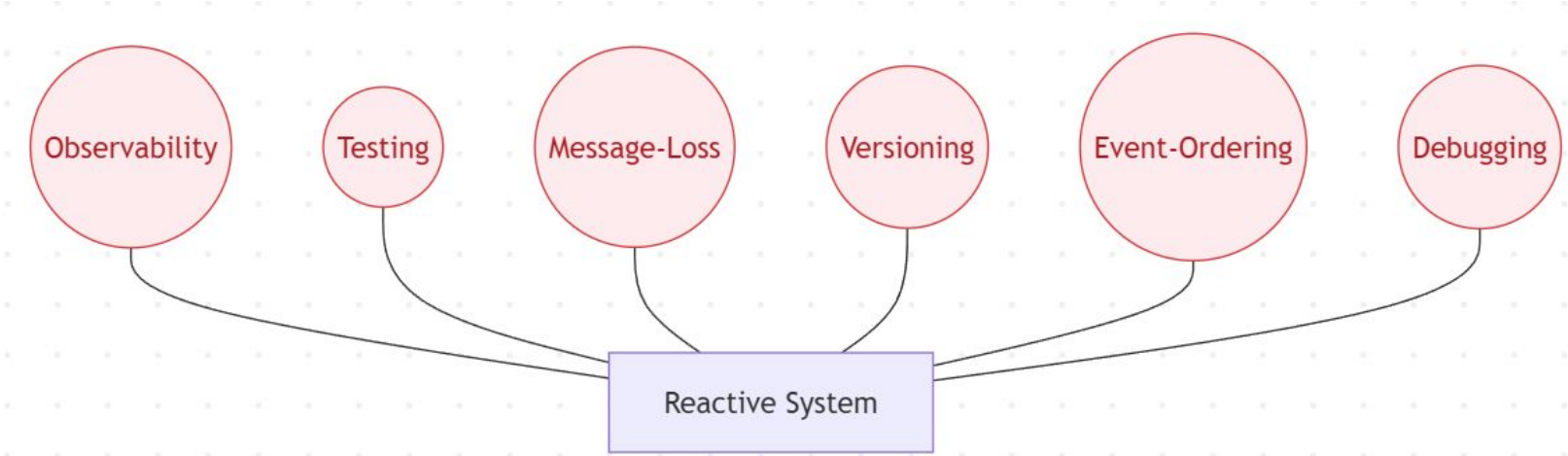
Real-World Examples

- Netflix: resilient streaming powered by reactive principles
- LinkedIn: message-driven systems (Kafka, Samza)
- Twitter: scalable, real-time updates via event-driven model
- Akka (Scala): actor-based reactive toolkit



Challenges of Reactive Systems

- Higher architectural complexity
- Debugging async flows is harder
- Tooling and mental model shift required
- Need for eventual consistency patterns



Summary: Reactive Principles

- Responsive = always reply in a timely fashion
- Resilient = isolate and recover from failure
- Elastic = adapt to load with scale
- Message-Driven = async communication & backpressure
- Together: a powerful approach to building modern systems
- What's next:
  - Distributed Consistency: Saga & Outbox Patterns

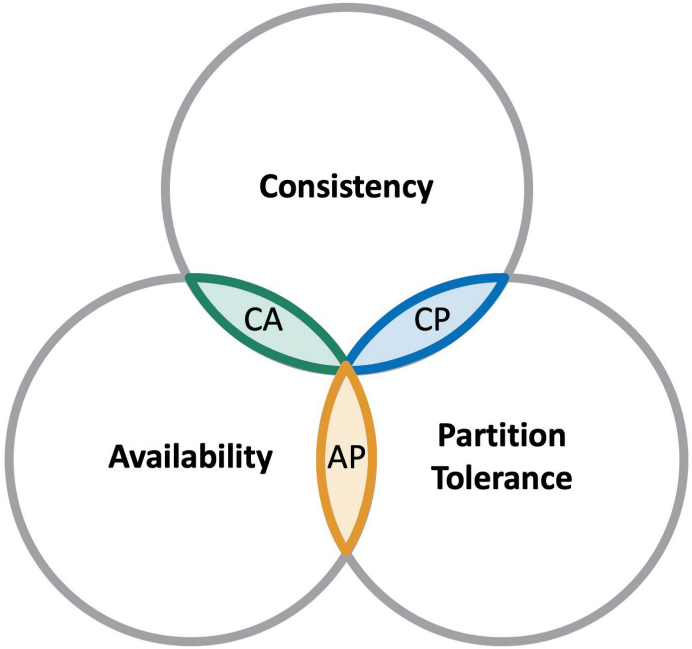
Distributed Consistency: Saga & Outbox Patterns

Event-Driven & Reactive Systems

Mastering Software Architecture Patterns and System Design

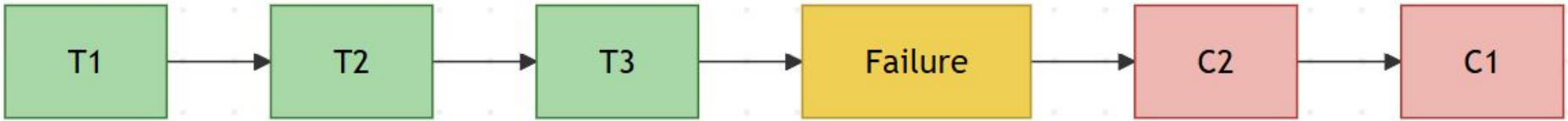
The Problem of Distributed Consistency

- ACID transactions are hard to scale across services
- Network delays, partial failures, retries = inconsistency risk
- CAP theorem: You must trade off consistency, availability, or partition tolerance
- Need loose coupling and eventual consistency



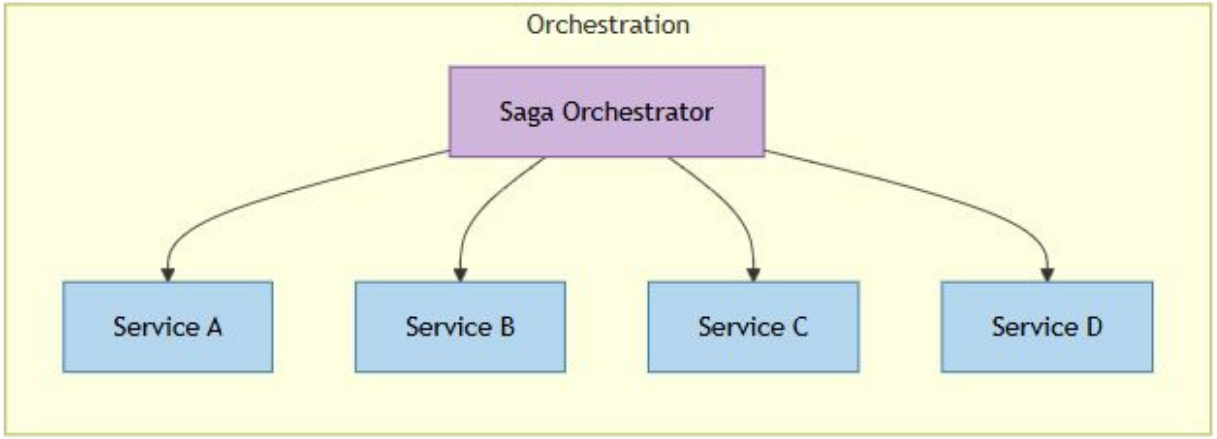
Sagas – A Distributed Transaction Alternative

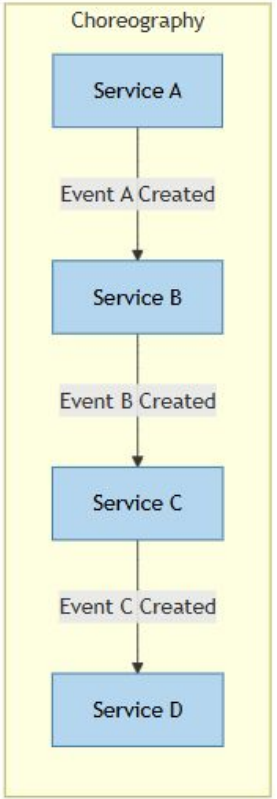
- A Saga breaks a transaction into a series of local steps
- Each step has a compensating action if something fails
- Works well for long-lived or multi-step workflows
- Supports eventual consistency, not isolation



Saga Execution Styles: Choreography vs Orchestration

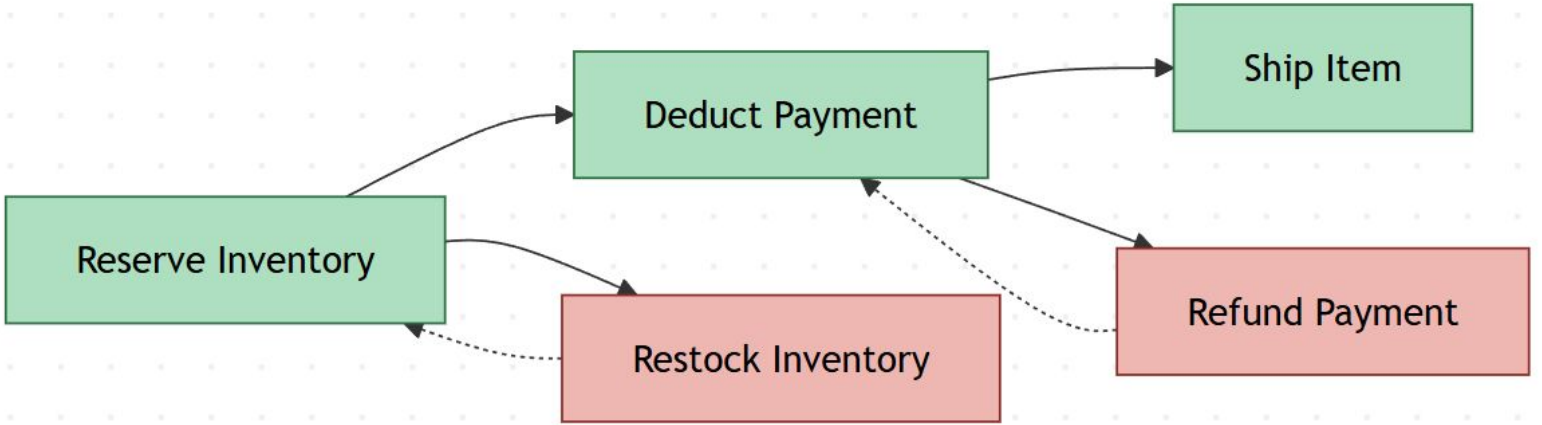
- Choreography: Services react to events autonomously
- Orchestration: Central controller directs the saga
- Choreography = decoupled but harder to trace
- Orchestration = centralized logic, easier observability





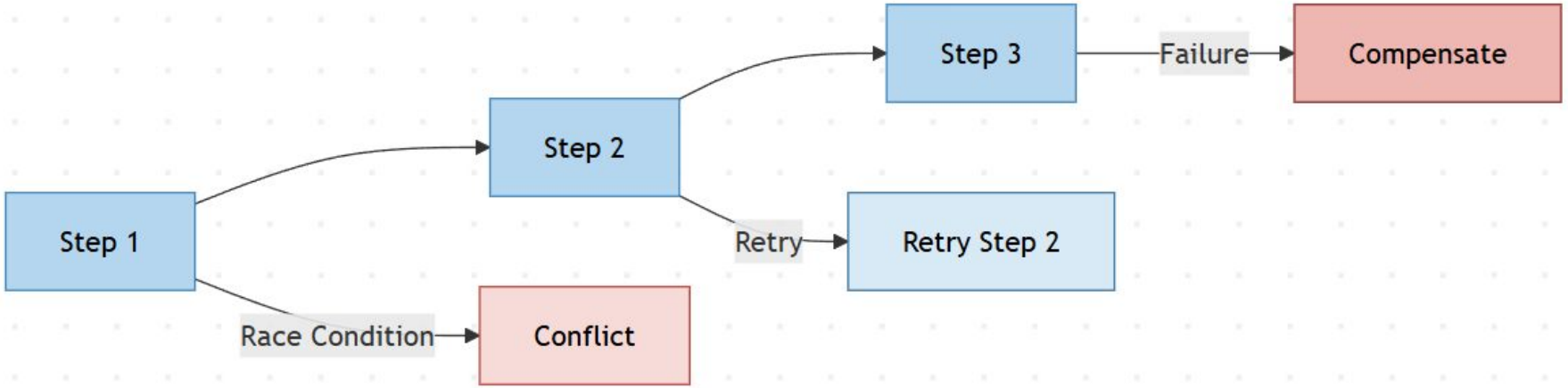
Real-World Saga Use Case

- Scenario: E-commerce order processing
- Steps: Reserve inventory → Deduct payment → Ship item
- Compensation: Restock item → Refund payment
- Demonstrates both forward and rollback logic



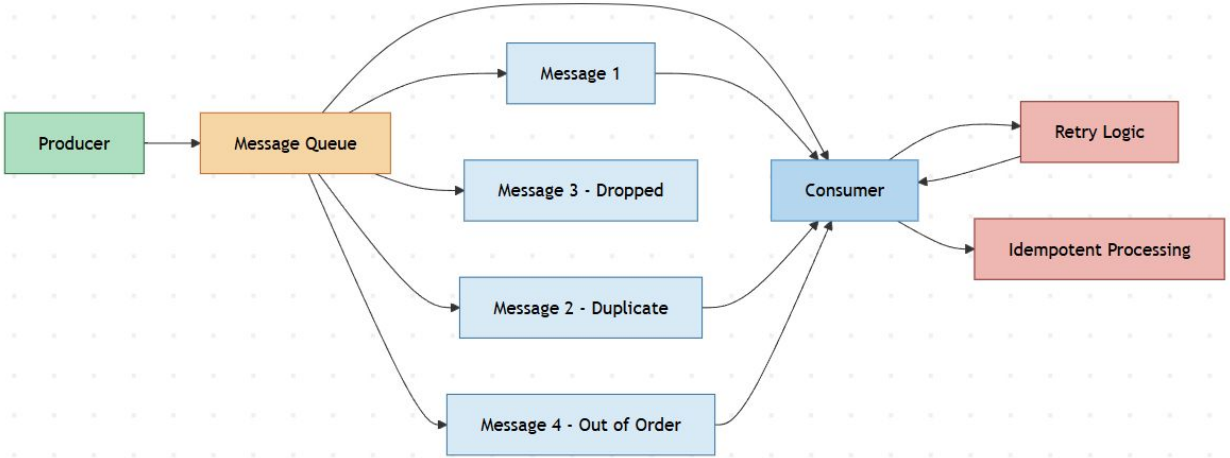
Challenges with Sagas

- Designing compensation logic can be complex
- No isolation: Concurrent updates may conflict
- Out-of-order or duplicate events possible
- Difficult to model global success/failure



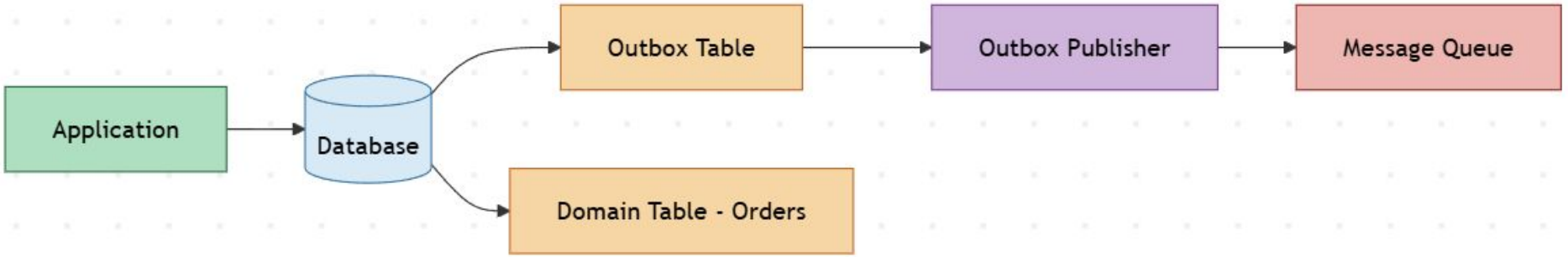
Reliable Messaging: The Missing Link

- Messaging is key to decoupling and async workflows
- But: messages can be lost, duplicated, or processed out of order
- Need at-least-once delivery + idempotent consumers
- Reliable delivery ensures saga steps are not skipped



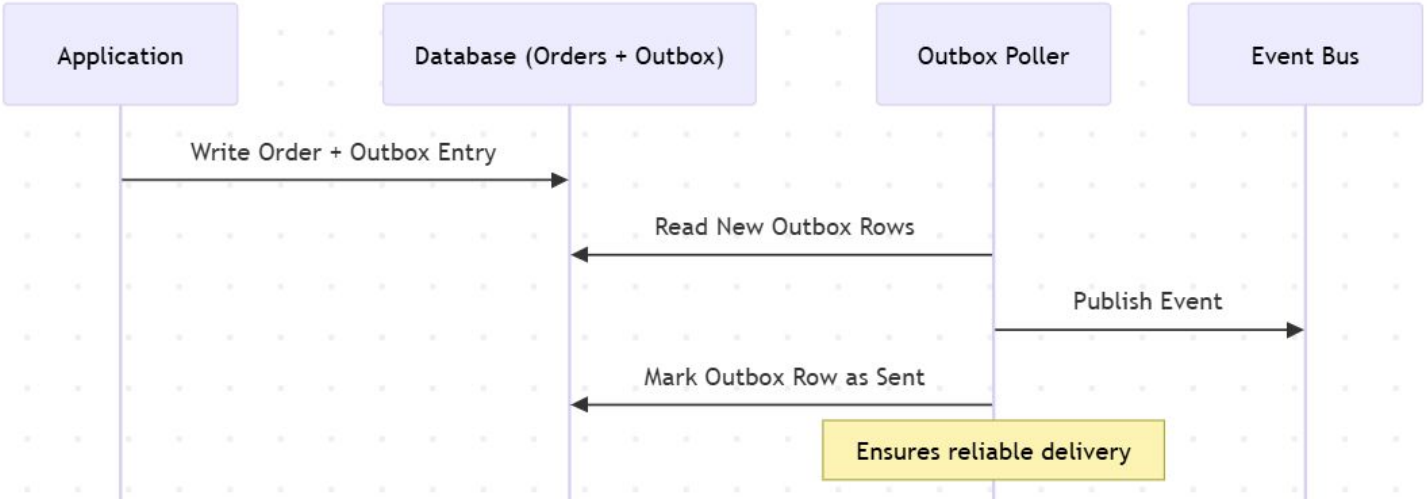
The Outbox Pattern – Ensuring Reliable Delivery

- Save domain changes + events in same local transaction
- A separate process reads from the outbox table and publishes events
- Guarantees atomicity: data change and event happen together
- Prevents lost messages



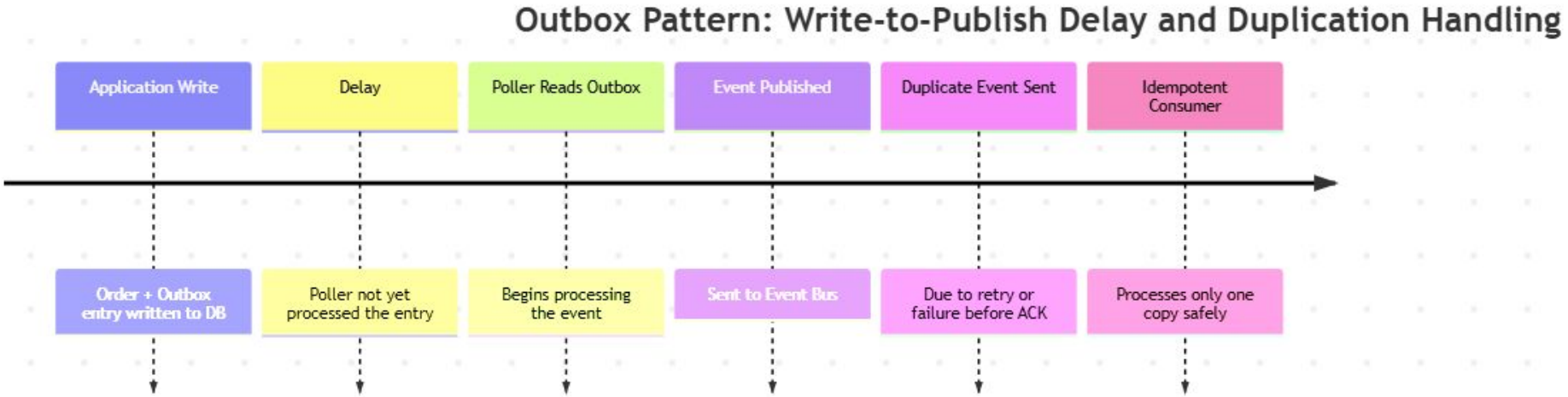
Outbox Pattern in Action

- Application writes to Orders and Outbox in same DB tx
- A background publisher polls the Outbox table
- Once published, message is marked as sent or deleted
- Works well with relational DBs



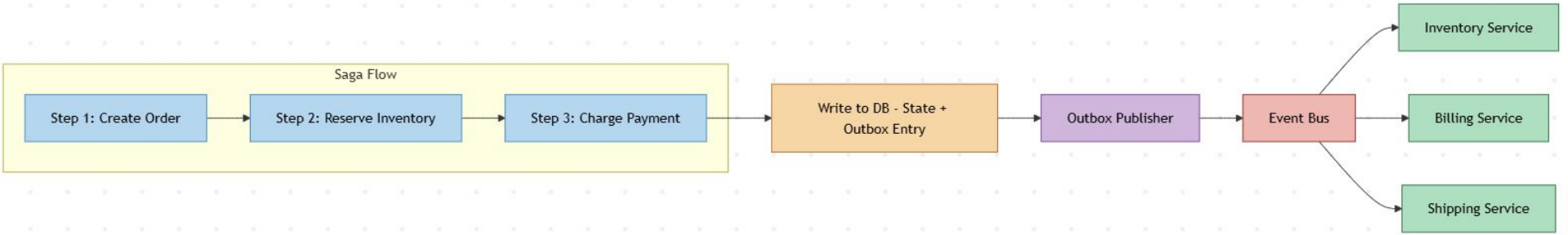
Challenges with the Outbox Pattern

- Duplicate messages still possible → idempotency required
- Polling delay = potential latency
- Adds DB load: outbox cleanup needed
- Requires coordination with message broker



Putting It All Together: Saga + Outbox

- Use Sagas for consistency, Outbox for reliable delivery
- Together, they handle multi-step logic and fault tolerance
- Crucial for microservice ecosystems and event-driven architectures
- Common in domains like payments, travel, logistics, and e-commerce



Summary

- ACID isn’t viable in distributed systems
- Saga = Coordinated local transactions + compensation
- Outbox = Reliable message publishing with atomic writes
- Design for eventual consistency, retries, and failure recovery
- What’s next:
  - Cloud-Native & Scalable Architecture Styles